2.1 Substrate 코어 구조 분석

개요

Substrate는 차세대 블록체인 혁신을 위한 프레임워크로, 모듈화된 아키텍처를 통해 개발자가 블록체인의 런타임 로직을 구축할 수 있도록 설계되었습니다. 이 문서에서는 `/substrate` 디렉토리의 구조를 상세히 분석하고, 각 주요 컴포넌트의 역할과 기능을 설명합니다.

Substrate 디렉토리 구조

```
substrate/
├── client/ # 클라이언트 사이드 구현체들
├── primitives/ # 핵심 데이터 타입과 인터페이스
├── frame/ # FRAME 팔렛 시스템
├── bin/ # 실행 가능한 바이너리들
├── utils/ # 유틸리티 라이브러리들
├── test-utils/ # 테스트 지원 도구들
├── docs/ # 문서화
├── scripts/ # 빌드 및 유지보수 스크립트
└── zombienet/ # 네트워크 테스트 설정
```

주요 컴포넌트 상세 분석

1. Client (`/substrate/client/`)

클라이언트는 Substrate 노드의 핵심 실행 환경을 제공하며, 블록체인 네트워크와의 상호작용을 담당합니다.

1.1 핵심 클라이언트 모듈

API (`client/api/`)

- **목적**: 클라이언트 인터페이스 정의
- **주요 기능**:
- 백엔드 추상화 인터페이스
- 호출 실행기 인터페이스
- 클라이언트 핵심 기능 정의

```
// 클라이언트 API 사용 예제
use sc_client_api::{Backend, CallExecutor, Client};


// 클라이언트 정보 조회
pub trait UsageProvider<Block: sp_runtime::traits::Block> {
fn usage_info(&self) -> ClientInfo<Block>;
}
```

Service (`client/service/`)

- **목적**: 네트워크, 클라이언트, 트랜잭션 풀 간 통신 관리
- **주요 기능**:
- 노드 서비스 시작 및 관리

- 컴포넌트 간 통신 조정
- 서비스 라이프사이클 관리

Database (`client/db/`)

- **목적**: 블록체인 데이터 저장 및 관리
- **주요 기능**:
- 블록 및 상태 데이터 저장
- 데이터베이스 백엔드 구현
- 성능 최적화된 스토리지

1.2 합의 메커니즘 (`client/consensus/`)

```
consensus/
├── aura/ # Aura 합의 알고리즘
├── babe/ # BABE 블록 생성
├── grandpa/ # GRANDPA 최종성
├── beefy/ # BEEFY 브릿지 최종성
├── manual-seal/ # 수동 블록 생성
└── pow/ # 작업 증명
```

BABE (Blind Assignment for Blockchain Extension)

- 블록 생성을 위한 VRF 기반 슬롯 할당
- 확률적 블록 생성 메커니즘

GRANDPA (GHOST-based Recursive ANcestor Deriving Prefix Agreement)

- 최종성 제공을 위한 비잔틴 합의

- 체인 분기 해결 및 최종 블록 확정

1.3 네트워킹 (`client/network/`)

```
network/
├── common/ # 공통 네트워크 유틸리티
├── sync/ # 블록 동기화
├── transactions/ # 트랜잭션 전파
├── statement/ # 스테이트먼트 처리
└── test/ # 네트워크 테스트
```

주요 기능:

- P2P 네트워크 통신
- 블록 및 트랜잭션 전파
- 피어 발견 및 관리
- 네트워크 프로토콜 구현

1.4 RPC 시스템 (`client/rpc*`)

```
rpc-related/
├── rpc/ # 핵심 RPC 구현
├── rpc-api/ # RPC API 정의
├── rpc-servers/ # RPC 서버 구현
└── rpc-spec-v2/ # RPC 스펙 v2 구현
```

2. Primitives (`/substrate/primitives/`)

Primitives는 Substrate 생태계 전반에서 사용되는 핵심 데이터 타입과 인터페이스를 정의합니다.

2.1 핵심 프리미티브

Core (`primitives/core/`)

- **목적**: 공유 가능한 Substrate 타입 정의
- **주요 구성요소**:
- 암호화 프리미티브 (sr25519, ed25519, ecdsa)
- 해시 타입 (H256, H160, H512)
- 큰 정수 타입 (U256, U512)

```
// sp-core 사용 예제
use sp_core::{
crypto::{Pair, Public},
sr25519,
H256,
U256,
};

// 키 페어 생성
let (pair, _seed) = sr25519::Pair::generate();
let  public = pair.public();

// 해시 타입 사용
let  hash: H256 = [0u8; 32].into();
```

```
// 큰 정수 연산
let value = U256::from(1000000);
```

Runtime (`primitives/runtime/`)

- **목적**: 런타임 실행 환경 정의
- **주요 기능**:
- 트랜잭션 타입 정의
- 블록 헤더 구조
- 런타임 API 인터페이스

API (`primitives/api/`)

- **목적**: 런타임 API 매크로 및 인터페이스
- **주요 기능**:
- API 버전 관리
- 런타임-클라이언트 통신 인터페이스

2.2 특화된 프리미티브

Consensus (`primitives/consensus/`)

- 합의 관련 데이터 구조
- 검증자 정보 및 합의 메시지

Storage (`primitives/storage/`)

- 스토리지 키 및 값 타입
- 스토리지 증명 구조

Transaction Pool (`primitives/transaction-pool/`)

- 트랜잭션 풀 인터페이스
- 트랜잭션 유효성 검증

3. FRAME (`/substrate/frame/`)

FRAME(Framework for Runtime Aggregation of Modularized Entities)은 Substrate 런타임 개발을 위한 모듈화된 시스템입니다.

3.1 시스템 팔렛

System (`frame/system/`)

- **목적**: 런타임의 기본 기능 제공
- **주요 기능**:
- 계정 관리
- 블록 정보 저장
- 이벤트 시스템

Timestamp (`frame/timestamp/`)

- **목적**: 블록 타임스탬프 관리
- **주요 기능**:
- 블록 시간 설정 및 검증
- 시간 기반 로직 지원

Balances (`frame/balances/`)

- **목적**: 계정 잔액 관리
- **주요 기능**:
- 토큰 전송
- 잔액 조회 및 관리

- 예약 잔액 처리

3.2 거버넌스 팔렛

Democracy (`frame/democracy/`)

- 민주적 의사결정 시스템
- 제안서 투표 및 실행

Council (`frame/collective/`)

- 위원회 기반 거버넌스
- 다중 서명 의사결정

Treasury (`frame/treasury/`)

- 재무 관리 시스템
- 자금 제안 및 지출 관리

3.3 스테이킹 및 합의

Staking (`frame/staking/`)

- **목적**: 지분 증명 시스템 구현
- **주요 기능**:
- 검증자 및 지명자 관리
- 보상 분배
- 슬래싱 메커니즘

Session (`frame/session/`)

- **목적**: 세션 관리 및 키 로테이션
- **주요 기능**:
- 검증자 세션 관리

- 키 변경 처리

4. 유틸리티 및 지원 도구

4.1 Bin (`/substrate/bin/`)

실행 가능한 바이너리들:

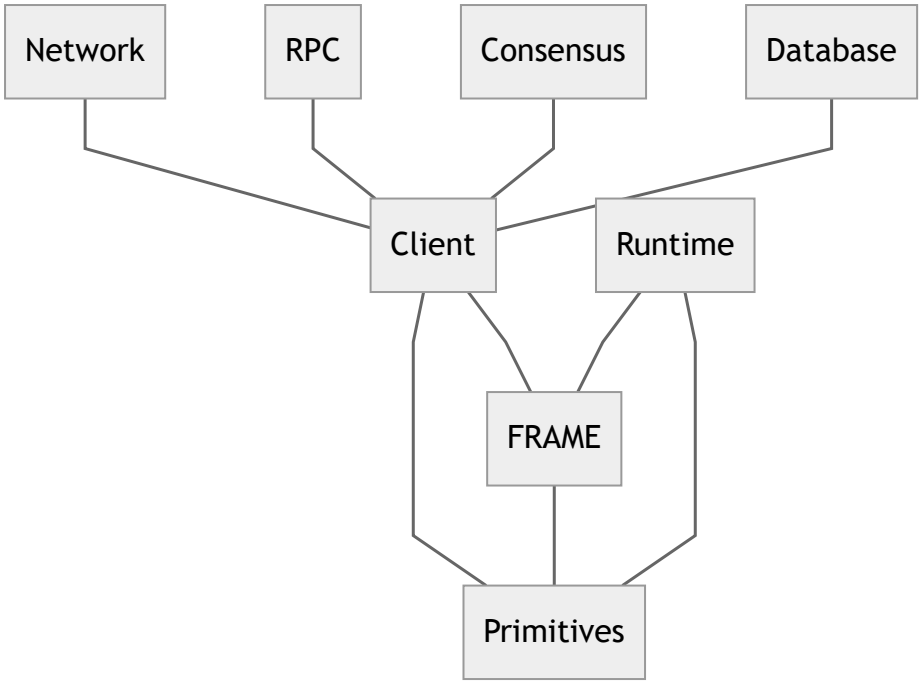
- **node/**: 예제 노드 구현
- **node-template/**: 노드 템플릿
- **utils/**: 유틸리티 도구들

4.2 Utils (`/substrate/utils/`)

개발 지원 유틸리티:

- **build-script-utils/**: 빌드 스크립트 도구
- **fork-tree/**: 포크 트리 자료구조
- **prometheus/**: 메트릭 수집

모듈 간 상호작용



공개 API 및 사용법 예제

클라이언트 API 사용

```
use sc_client_api::{Backend, BlockBackend, HeaderBackend};
use sp_runtime::traits::Block as BlockT;

// 블록 정보 조회
async fn get_block_info<B, Block>(
client: &B,
hash: Block::Hash,
) -> Result<Option<Block>, B::Error>
where
B: BlockBackend<Block>,
Block: BlockT,
```

```
{
client.block(hash)
}
```

프리미티브 사용

```
use sp_core::{crypto::Pair, sr25519};
use sp_runtime::{
traits::{BlakeTwo256, Hash},
AccountId32,
};

// 계정 생성
let (pair, _) = sr25519::Pair::generate();
let  account: AccountId32 = pair.public().into();

// 해시 계산
let  data = b"Hello, Substrate!";
let  hash = BlakeTwo256::hash(data);
```

FRAME 팔렛 사용

```
use frame_support::{
```

```
construct_runtime,

traits::{Get, ConstU32},

};

use frame_system as system;



// 런타임 구성

construct_runtime!(

pub  struct Runtime {

System: system,

Balances: pallet_balances,

// 추가 팔렛들...

}

);



// 팔렛 설정

impl system::Config for Runtime {

type RuntimeOrigin = RuntimeOrigin;

type RuntimeCall = RuntimeCall;

type Hash = sp_core::H256;

type Hashing = sp_runtime::traits::BlakeTwo256;

type AccountId = sp_runtime::AccountId32;

type Block = Block;

type BlockHashCount = ConstU32<2400>;
```

```
// 추가 설정...

}
```

라이센스 구조

Substrate는 이중 라이센스 구조를 가집니다:

- **Apache 2.0**: Primitives (`sp-*`), FRAME (`frame-*`), 팔렛들, 바이너리
- **GPL v3.0**: Client (`sc-*`) - 클래스패스 링킹 예외 포함

이러한 구조는 대부분의 팀이 Apache2 라이센스 코드에서만 작업할 수 있도록 하여 상업적 자유도를 보장하면서도, 핵심 로직 개선사항은 커뮤니티에 기여하도록 장려합니다.

다음 단계

이 문서에서 다룬 Substrate 코어 구조를 바탕으로, 다음 문서들에서는 각 컴포넌트의 세부 구현과 사용법을 더 자세히 살펴보겠습니다:

- 2.2 FRAME 팔렛 시스템 상세 분석
- 2.3 런타임 구성 및 개발 가이드
- 2.4 합의 메커니즘 (BABE, GRANDPA) 구현 분석